choosing filters

Table of contents

packages required 1
data prep 1
analyse of the filtered datasets 3
Advanced visualisation of the data generated 5

packages required

This analysis depends on the following packages:

data prep

the dataset is loaded and curred. We remove adult = 0 and NAs

```
data250_apisum_00 <- read.csv("../../../../../../01.data/2023_screen_specifics_251genotypes/0
  mutate(ivip = as.factor(ivip)) %>%
  mutate(session = as.factor(session))



data250_apisum_01 <- data250_apisum_00 %>% filter(!is.na(larvae_total_number))
data250_apisum_01$ivip <- droplevels(data250_apisum_01$ivip)

data250_apisum_02<- data250_apisum_01 %>% filter(adult_alive != "0") #remove if no adult
data250_apisum_02$ivip <- droplevels(data250_apisum_02$ivip)

data250_apisum_02$ivip <-as.factor(data250_apisum_02$ivip)
```

Once we have removed the NAs and all rows for which we don't have an adult, we can still make adjustments to the dataset. We can remove outliers to ensure that our distribution approaches a normal distribution, and we can also filter out genotypes based on a threshold for the minimum number of repetitions required to keep a genotype. We have 3 methods to handle outliers: 1)keep the outliers; 2)remove manually the outliers based on biological reasons; 3) use the interquartile range method (IQR) to remove all data outside a specific range.

The goal of this script is to study how our model reacts to changes in the outlier removal method and the minimum number of repetitions required to retain a genotype.

in the folowing loop we varry two parameters, i the minimal number of reps for the genotypes and the outlier filtering method.

function to calculate all metric used to compare filters

```
analyze_filtered_data <- function(data, i, filter_name) {
  data <- data %>%
    group_by(ivip) %>%
    filter(n() >= i) %>%
    ungroup() %>%
    droplevels()

  if (nrow(data) < 5 || nlevels(data$ivip) < 2) {
    return(NULL)  # skip iterations with too little data
  }

  NGEN <- nlevels(data$ivip)
  n_after <- nrow(data)

  # Fit the model
  mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)
  residuals_values <- residuals(mod)
  RMSE <- sqrt(mean(residuals_values^2))
  shapiro_res <- shapiro.test(residuals_values)$p.value
  shapiro <- shapiro.test(data$larvae_total_number)$p.value

  # Heritability model
  mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)
  estimates <- as.data.frame(VarCorr(mod_h2))
  genetic_variance <- estimates$vcov[1]
  residual_variance <- estimates$vcov[3]
  nbE <- nlevels(data$session)
  H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))
```

```
  return(data.frame(
    filter = filter_name, i = i, RMSE = RMSE, n_after = n_after,
    H2 = H2, shapiro = shapiro, shapiro_res = shapiro_res, NGEN = NGEN
  ))
}
```

analyse of the filtered datasets

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu


###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_apisum_02$larvae_total_number)
    lower_bound <- quantile(data250_apisum_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_apisum_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data_iqr <- data250_apisum_02[data250_apisum_02$larvae_total_number >= lower_bound & data


for (i in 2:9) {
  res <- analyze_filtered_data(data_iqr, i, filter_name = "IQR")
  if (!is.null(res)) results <- rbind(results, res)
}

for (i in 2:9) {
  res <- analyze_filtered_data(data250_apisum_02, i, filter_name = "no_filter")
  if (!is.null(res)) results <- rbind(results, res)
    }

data_manual<- data250_apisum_02 %>% filter(larvae_total_number != "0")

for (i in 2:9) {
  res <- analyze_filtered_data(data_manual, i, filter_name = "Manual_filter")
  if (!is.null(res)) results <- rbind(results, res)
}
```

```
print(results)
```

```
          filter i     RMSE n_after        H2      shapiro  shapiro_res NGEN
1            IQR 2 6.657805    2260 0.4108407 5.820174e-07 3.813894e-06  243
2            IQR 3 6.666687    2248 0.4112436 6.649820e-07 4.567990e-06  237
3            IQR 4 6.690371    2215 0.4113008 7.202519e-07 6.179531e-06  226
4            IQR 5 6.682081    2191 0.4114434 1.052337e-06 4.305833e-06  220
5            IQR 6 6.684478    2176 0.4136635 1.174087e-06 3.925160e-06  217
6            IQR 7 6.691100    2128 0.4163125 1.538742e-06 9.974474e-06  209
7            IQR 8 6.695692    2051 0.4062488 2.121472e-06 2.534664e-06  198
8            IQR 9 6.696062    1851 0.4047060 3.141034e-06 1.104593e-05  173
9      no_filter 2 7.481163    2343 0.3705465 3.980241e-11 1.796814e-14  243
10     no_filter 3 7.492743    2331 0.3716888 3.336373e-11 2.262427e-14  237
11     no_filter 4 7.519257    2301 0.3714715 4.682873e-11 3.553872e-14  227
12     no_filter 5 7.516380    2273 0.3709082 3.906445e-11 3.071967e-14  220
13     no_filter 6 7.516380    2273 0.3709082 3.906445e-11 3.071967e-14  220
14     no_filter 7 7.555582    2231 0.3690504 1.384709e-10 5.371531e-14  213
15     no_filter 8 7.568168    2154 0.3636776 5.928548e-10 3.184938e-14  202
16     no_filter 9 7.612903    2018 0.3636364 9.730073e-10 8.508649e-14  185
17 Manual_filter 2 7.317314    2332 0.3832690 3.836452e-11 1.785912e-12  243
18 Manual_filter 3 7.328357    2320 0.3844173 3.176391e-11 2.268226e-12  237
19 Manual_filter 4 7.353900    2290 0.3841713 4.205613e-11 3.497580e-12  227
20 Manual_filter 5 7.350153    2262 0.3837297 3.918677e-11 2.875336e-12  220
21 Manual_filter 6 7.350153    2262 0.3837297 3.918677e-11 2.875336e-12  220
22 Manual_filter 7 7.386433    2220 0.3820155 1.447377e-10 5.260479e-12  213
23 Manual_filter 8 7.375090    2136 0.3814073 8.414128e-10 5.034994e-12  201
24 Manual_filter 9 7.437219    2008 0.3775566 1.157443e-09 1.057466e-11  185
```

Based on these results it seems appropriate to select the IQR filter as it yield best heritability, lower RMSE and more normal data.

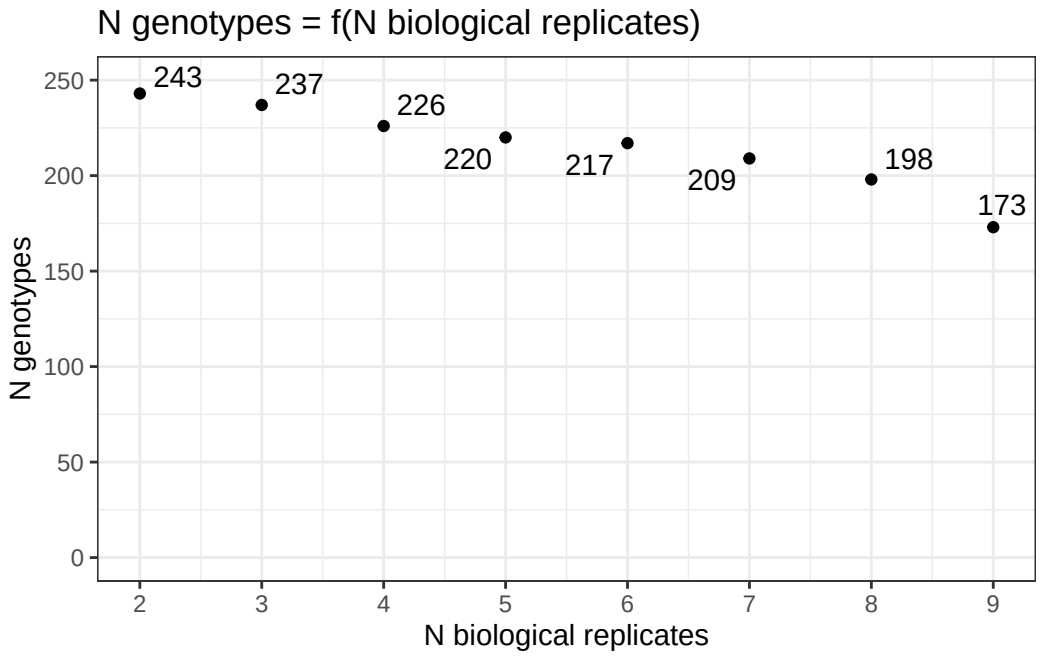
```
write.csv(x = results,
          file = "filter_annalysis.csv",
          quote = F, row.names = F)
```

```
iqr_results <- results %>% filter(filter == "IQR")
```

We now need to visualize the data we generated from the IQR filter to choose the best value for i (minimal number of repetitions to keep a genotype)

Advanced visualisation of the data generated

```
ggplot(iqr_results, aes(x = i, y = NGEN)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,250)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "N biological replicates", y = "N genotypes") +
  ggtitle("N genotypes = f(N biological replicates)")
```



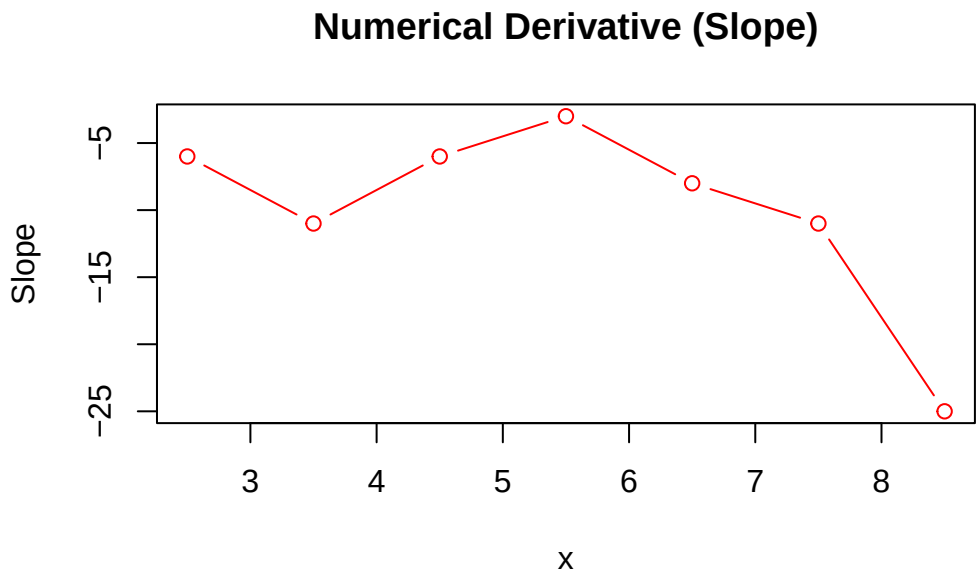

```
y<-as.vector(iqr_results$NGEN)
x<-as.vector(iqr_results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
```

```r
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



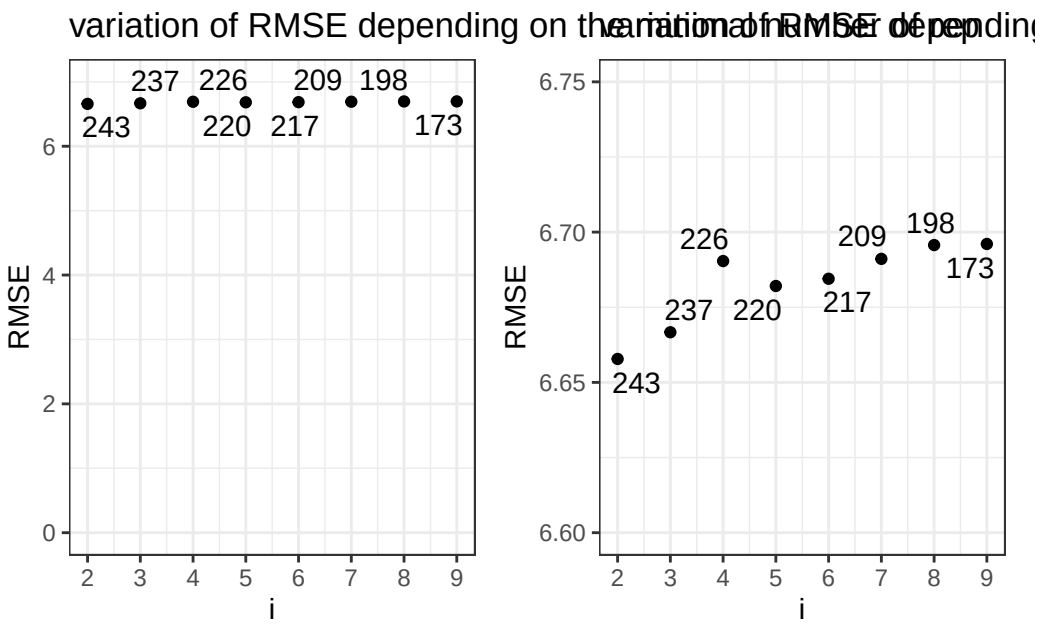
Based on the variation in genotype number the best i value seems to be 6

```r
away <- ggplot(iqr_results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,7)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "i", y = "RMSE") +
  ggtitle("variation of RMSE depending on the minimal number of rep")

close<- ggplot(iqr_results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(6.6,6.75)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "i", y = "RMSE") +
```

```
  ggtitle("variation of RMSE depending on the minimal number of rep")

away+close
```

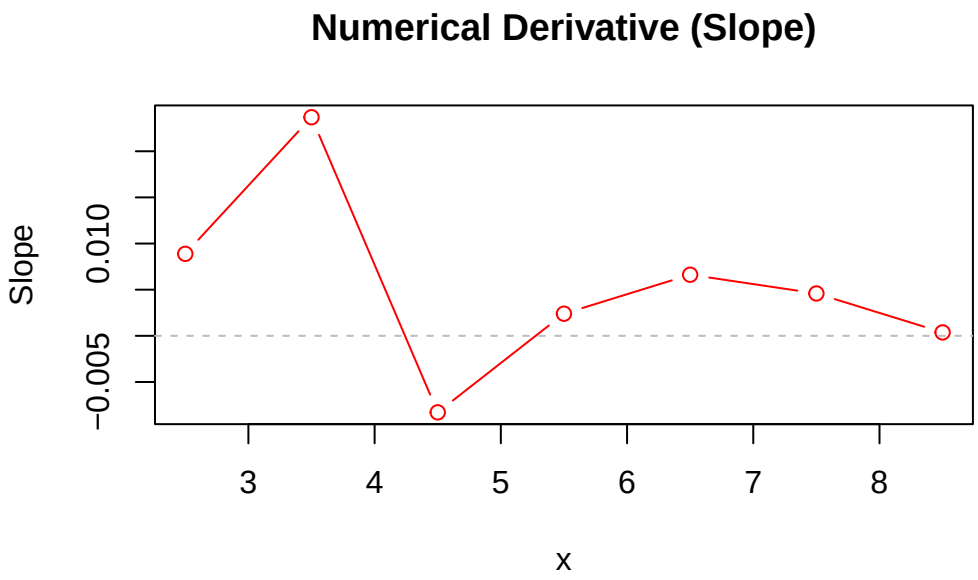



```
y<-as.vector(iqr_results$RMSE)
x<-as.vector(iqr_results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

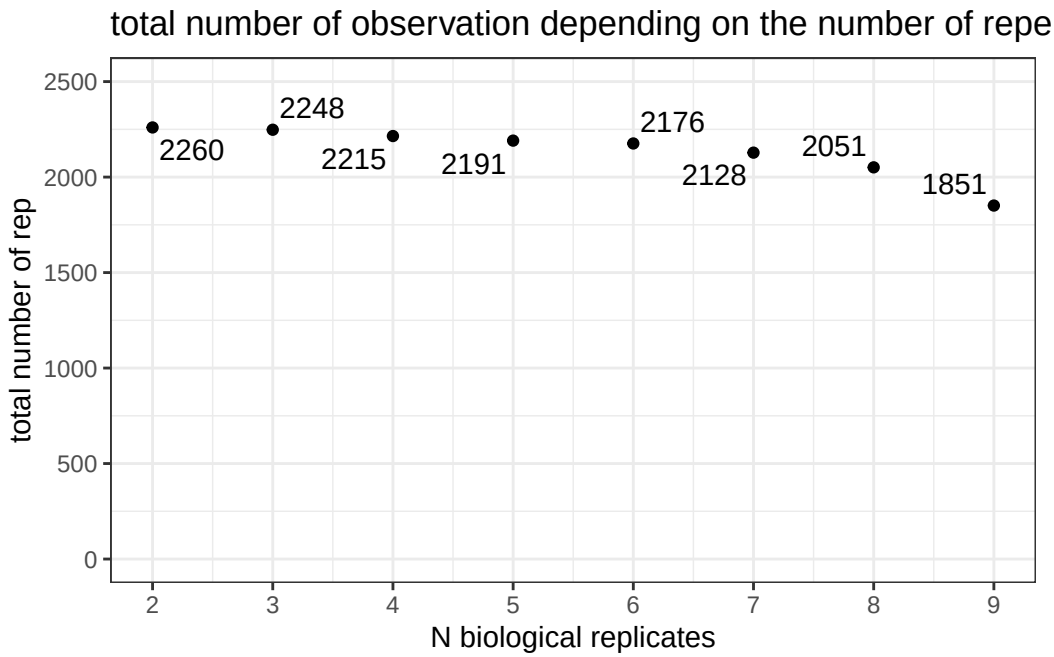
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



it is complicated to select a value for i using RMSE as the value don't vary much

```
ggplot(iqr_results, aes(x = i, y = n_after)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,2500)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = n_after), arrow = T) +
  labs(x = "N biological replicates", y = "total number of rep") +
  ggtitle("total number of observation depending on the number of repetitions")
```

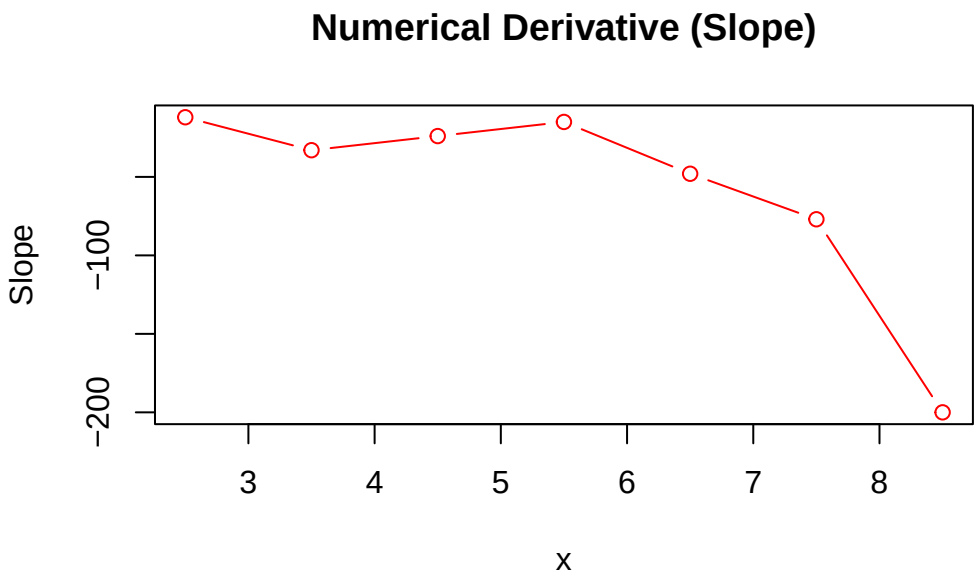



```r
y<-as.vector(iqr_results$n_after)
x<-as.vector(iqr_results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



To maximise the overall number of data the best i value seems to be 6

```
close<-ggplot(iqr_results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0.4, 0.42)) +
  theme_bw() +
  labs(
    x = "N biological replicates",
    y = "Total number of rep"
  ) +
  ggtitle("H2 depending on the number of repetitions")

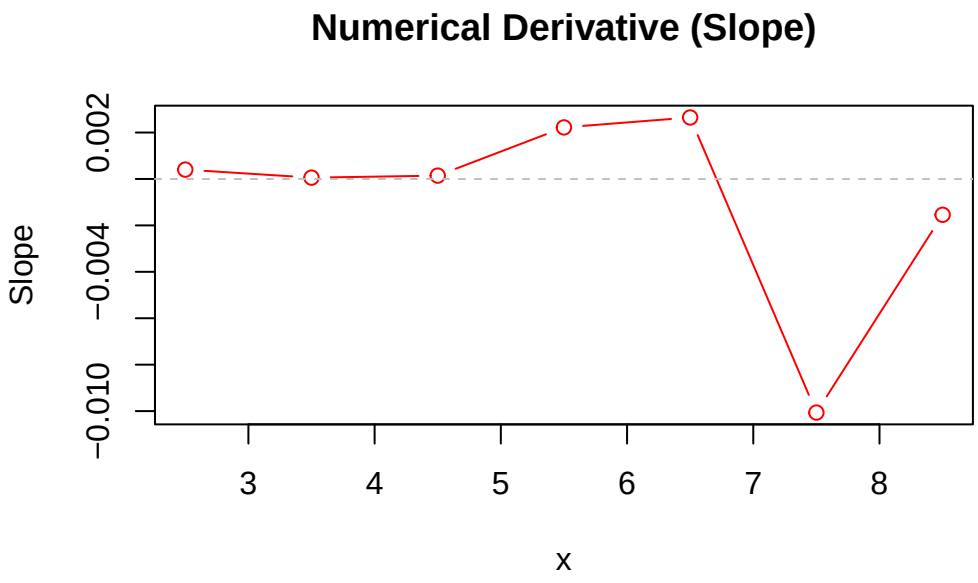
away<-ggplot(iqr_results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0, 0.5)) +
  theme_bw() +
  labs(
    x = "N biological replicates",
    y = "Total number of rep"
  ) +
  ggtitle("H2 depending on the number of repetitions")
```

```r
y<-as.vector(iqr_results$H2)
x<-as.vector(iqr_results$i)

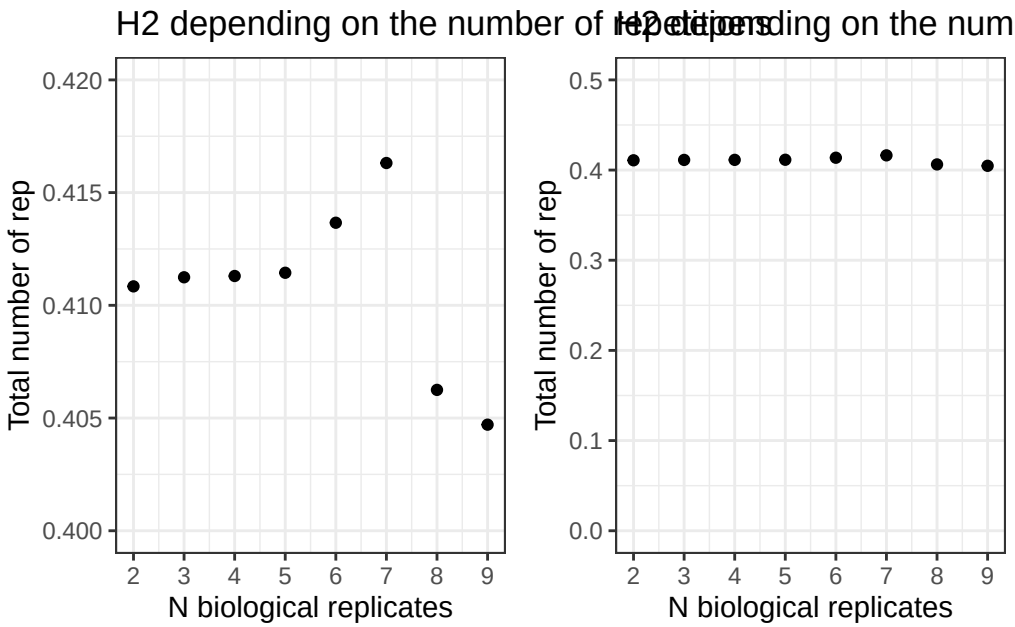
# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```




```r
close + away
```



to maximise heritability the best value for i is 7, and the second best value is 6